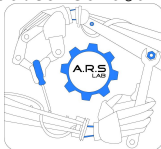


# Basics of Prolog

Corrado Santoro

Dipartimento di Matematica e Informatica  
`corrado.santoro@unict.it`



Corso di “Introduction to Robotics and AI”

# Elementi di PROLOG

- Il **PROLOG** (PROgramming in LOGic) è un linguaggio di programmazione simbolico basato su predicati logici
- Oltre ai classici tipi numerici, il PROLOG introduce il concetto di **atomo**: esso è un **letterale** che inizia per lettera **minuscola**, ed è un **simbolo indivisibile** (non è una “stringa”!)
- Un letterale che inizia per lettera **Maiuscola** è invece una **variabile**:
  - **cube** è un atomo
  - **Cube** è una variabile

# Rappresentazione tramite “fatti” (Beliefs)

## Fatti o Beliefs

In PROLOG, La base di conoscenza è formata da **fatti certi**, i quali sono rappresentati tramite formule atomiche

## Esempi

- Cilindro, prisma e sfera sono dei blocchi:  
`block(cylinder)`  
`block(prism)`  
`block(sphere)`
- Il cilindro è rosso, il prisma è bianco, la sfera è nera:  
`color(cylinder, red)`  
`color(prism, white)`  
`color(sphere, black)`
- Il cilindro è sul tavolo 1, il prisma e la sfera sono sul tavolo 2:  
`upon(cylinder, desk1)`  
`upon(prism, desk2)`  
`upon(sphere, desk2)`

# Rappresentazione tramite “fatti” (Beliefs)

## Base di Conoscenza

Per popolare la base di conoscenza, è sufficiente utilizzare lo statement PROLOG **assert**

```
?- assert(block(cylinder)).  
true.
```

```
?- assert(block(prism)).  
true.
```

```
?- assert(upon(prism,green_desk)).  
true.
```

```
?-
```

# Interrogazioni e Conoscenza Derivata

## Interrogazioni sulla Base di Conoscenza

La base di conoscenza può essere interrogata per sapere se un determinato fatto è vero o falso.

```
?- block(cylinder) .  
true.
```

```
?- block(cube) .  
false.
```

```
?- block(sphere) .  
true.
```

# Interrogazioni e Conoscenza Derivata

## Interrogazioni sulla Base di Conoscenza

La base di conoscenza può essere interrogata anche usando variabili **quantificate universalmente**

```
?- assert(block(cylinder)).  
true.
```

```
?- assert(block(prism)).  
true.
```

```
?- assert(block(sphere)).  
true.
```

```
?- block(X).  
X = cylinder ;  
X = prism ;  
X = sphere.
```

# Interrogazioni e Conoscenza Derivata

## Interrogazioni sulla Base di Conoscenza

La base di conoscenza può essere interrogata usando variabili **quantificate universalmente**

```
?- block(X).  
X = cylinder ;  
X = prism ;  
X = sphere.
```

L'interrogazione **block(x)** equivale a:

$$\forall x : \text{block}(x)$$

# Interrogazioni e Conoscenza Derivata

## Interrogazioni sulla Base di Conoscenza

E' possibile combinare le interrogazioni utilizzando la “,” che ha ruolo di connettivo **AND**:

```
?- block(Obj),color(Obj,Col) .  
Obj = cylinder,  
Col = red ;  
Obj = prism,  
Col = white ;  
Obj = sphere,  
Col = black.  
false.  
  
?-
```

**Tutti gli oggetti, con il relativo colore**

# Interrogazioni e Conoscenza Derivata

## Interrogazioni sulla Base di Conoscenza

E' possibile combinare le interrogazioni utilizzando la “,” che ha ruolo di connettivo **AND**:

```
?- block(Obj), upon(desk2, Obj), color(Obj, Col) .  
Obj = prism,  
Col = white ;  
Obj = sphere,  
Col = black.  
false.  
  
?-
```

**Tutti gli oggetti, con il relativo colore, che si trovano sul “desk2”**

# Interrogazioni e Conoscenza Derivata

## Conoscenza Derivata

Tramite clausole del primo ordine è possibile definire nuovi predicati e derivare nuova conoscenza.

Esempio: definiamo il predicato “oggetto nero”:

```
black_object(X) :- block(X), color(X, black).
```

```
?- black_object(Obj).
```

```
Obj = sphere.
```

```
?-
```

La definizione equivale all'implicazione logica:

$$\forall x : \text{block}(x) \wedge \text{color}(x, \text{black}) \Rightarrow \text{black\_object}(x)$$

# Interrogazioni e Conoscenza Derivata

## Conoscenza Derivata

Esempio: definiamo il predicato “oggetto nero”:

```
black_object(X) :- block(X), color(X, black).
```

```
?- black_object(Obj).
```

```
Obj = sphere.
```

```
?-
```

Oppure:

$$\forall x : \textit{black\_object}(x) \Leftarrow \textit{block}(x) \wedge \textit{color}(x, \textit{black})$$

In realtà il simbolo PROLOG “:-” è una stilizzazione del simbolo “”

# Interrogazioni e Conoscenza Derivata

## Conoscenza Derivata e Negazione

E' anche possibile inserire negazioni.

Esempio: definiamo il predicato “tavolo libero” il quale è **true** se non c'è alcun oggetto su di esso:

```
free_desk(Desk) :- \+upon(Desk, _).
```

```
?- free_desk(desk1).  
false.
```

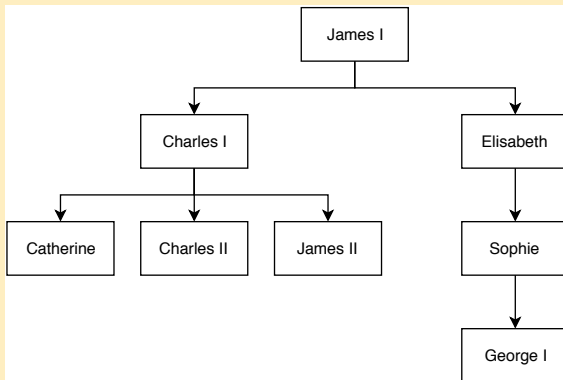
```
?- free_desk(desk2).  
false.
```

```
?- free_desk(desk3).  
true.
```

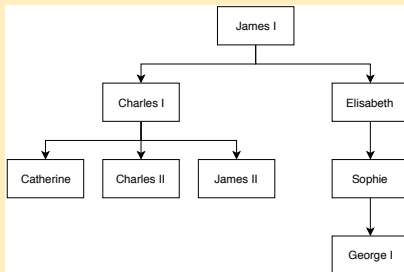
Equivalente a:

$$\text{free\_desk}(x) \Leftarrow \nexists y : \text{upon}(y, x)$$

# Esempio: L'Albero Genealogico



# Esempio: L'Albero Genealogico



## Fatti

- **male(X)**  $\rightarrow$  X è un uomo
- **female(X)**  $\rightarrow$  X è una donna
- **parent(X, Y)**  $\rightarrow$  X è genitore di Y

# Esempio: L'Albero Genealogico

## Fatti

```
:-  
    assert(male(james1)),  
    assert(male(charles1)),  
    assert(male(charles2)),  
    assert(male(james2)),  
    assert(male(georgel)),  
  
    assert(female(catherine)),  
    assert(female(elizabeth)),  
    assert(female(sophia)),  
  
    assert(parent(james1, charles1)),  
    assert(parent(james1, elizabeth)),  
    assert(parent(charles1, charles2)),  
    assert(parent(charles1, catherine)),  
    assert(parent(charles1, james2)),  
    assert(parent(elizabeth, sophia)),  
    assert(parent(sophia, georgel)).
```

# Esempio: L'Albero Genealogico

## Conoscenza Derivata

- **father**(X, Y)  $\rightarrow$  X è il padre di Y:

$$\forall x, y : \text{parent}(x, y) \wedge \text{male}(x) \Rightarrow \text{father}(x, y)$$

- **mother**(X, Y)  $\rightarrow$  X è la madre di Y:

$$\forall x, y : \text{parent}(x, y) \wedge \text{female}(x) \Rightarrow \text{mother}(x, y)$$

```
father(X,Y) :- parent(X,Y), male(X).
```

```
mother(X,Y) :- parent(X,Y), female(X).
```

# Esempio: L'Albero Genealogico

## Conoscenza Derivata

- **sibling**(X, Y)  $\rightarrow$  X è fratello o sorella di Y (X e Y hanno lo stesso genitore):

$$\exists p, \forall x, y : \text{parent}(p, x) \wedge \text{parent}(p, y) \wedge x \neq y \Rightarrow \text{sibling}(x, y)$$

```
sibling(X,Y) :- parent(P, X), parent(P, Y), X \= Y.
```

# Esempio: L'Albero Genealogico

## Conoscenza Derivata

- **brother**(X, Y)  $\rightarrow$  X è fratello di Y:

$$\forall x, y : sibling(x, y) \wedge male(x) \Rightarrow brother(x, y)$$

- **sister**(X, Y)  $\rightarrow$  X è sorella di Y:

$$\forall x, y : sibling(x, y) \wedge female(x) \Rightarrow sister(x, y)$$

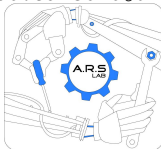
```
brother(X,Y) :- sibling(X,Y), male(X).  
sister(X,Y) :- sibling(X,Y), female(X).
```

# Basics of Prolog

Corrado Santoro

Dipartimento di Matematica e Informatica

`corrado.santoro@unict.it`



Corso di “Introduction to Robotics and AI”